

Every harvest, on the record.

The complete guide: what Seed2Store is, how growers and buyers use it end to end, and how it's built, from MongoDB to the certificate contract.



The image shows a screenshot of the Seed2Store web application. The interface is dark-themed with green accents. At the top, there's a navigation bar with the Seed2Store logo, links for Marketplace, Auctions, Ledger, and Verify, and a 'Connect wallet' button. The main content area features a large heading 'Every harvest, on the record.' followed by a paragraph explaining the process: 'Growers list lots with a tamper-evident certificate of origin, then sell direct: buy-now, sealed offers or open auctions. Buyers see exactly where, when and how it was grown, from seed to store.' Below this are two buttons: 'Browse the market →' and 'List your harvest'. To the right, there's a 'CERTIFICATE OF ORIGIN' for 'wheat' with a QR code and a circular seal. The certificate details include: Grown by 0xdce8...f6a5, COMMODITY: Wheat, LOT SIZE: 25 kg, GRADE: Ungraded, and HARVESTED: Certified. Below the certificate, there's a 'LIVE' auction for 'wheat' with a price of \$5,000 and 0 bids. At the bottom, there's a table of lots. The first lot is 'wheat' with a price of \$5,000 and 23h 89m. The second lot is '1121 Basmati, Aged 12 Months' with a price of \$7,500 and 23h 34m. The footer contains the text 'Fresh off the field, with papers.' and an 'All lots →' button.

Seed2Store Marketplace Auctions Ledger Verify Connect wallet

Every harvest, on the record.

Growers list lots with a tamper-evident certificate of origin, then sell direct: buy-now, sealed offers or open auctions. Buyers see exactly where, when and how it was grown, from seed to store.

[Browse the market →](#) [List your harvest](#)

CERTIFICATE OF ORIGIN
S2S-C7BB-4FCD

wheat
Grown by 0xdce8...f6a5

COMMODITY	ORIGIN	HARVESTED
Wheat	—	—
LOT SIZE	GRADE	ORGANIC
25 kg	Ungraded	Certified

keccak256 8xc7bb4fd77...f29483c56c Registered Oct 8, 2026

LIVE · 23h 89m
wheat
\$5,000 · 0 bids

wheat	\$5,000 23h 89m	1121 Basmati, Aged 12 Months Amritsar, Punjab	\$7,500 23h 34m
--------------	---------------------------	---	---------------------------

Fresh off the field, with papers. [All lots →](#)

A market that keeps **the papers.**

Grain, coffee and produce still trade through phone calls, middlemen and paper slips. Origin data, quality grades and fair prices get lost at every hand-off. Seed2Store keeps the record with the lot.

GROWERS

List and certify

Register a lot with crop, origin, harvest date, grade and photos. Its provenance is fingerprinted with keccak-256 and can be minted as an ERC-721 certificate.

BUYERS

Judge, then trade

See where, when and how it was grown. Buy outright, send a private offer, or bid in an open auction with anti-sniping.

ANYONE

Check the papers

Paste a serial like `s2s-695D-EB66` to confirm the record is unchanged and see who holds the certificate today.

Contents

03 Two sides, one market

04 The grower's journey

05 The buyer's journey

06 The sale, and after

07 Verification

08 Architecture

09 MongoDB & the contract

10 API reference

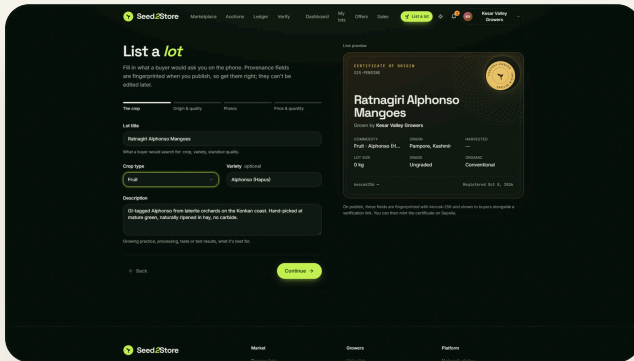
11 Setup, config & deploy

12 Security & troubleshooting

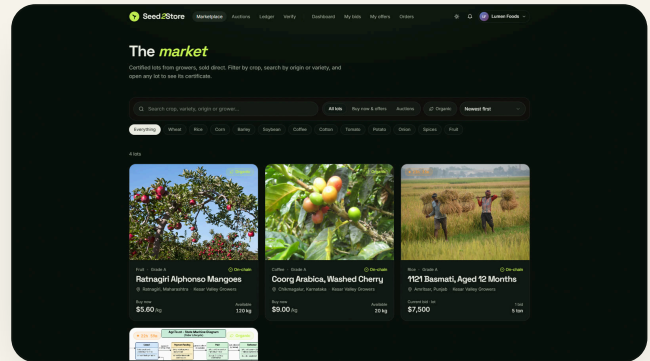
Runs with **zero configuration**. A burner wallet and local storage work out of the box; set `SEED2STORE` for a sample market. MongoDB and a real chain switch on with a few environment variables.

Two sides, one market.

Every account is a wallet. One switch in the account menu flips you between buyer and farmer mode, so the same person can sell a harvest and buy inputs.



GROWER Kesar Valley Growers, Pampore, lists a lot of Kashmiri saffron.



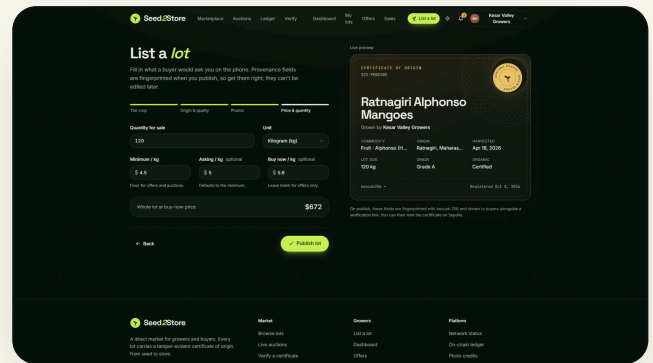
BUYER Lumen Foods, Bengaluru, browses certified lots.

Signing in

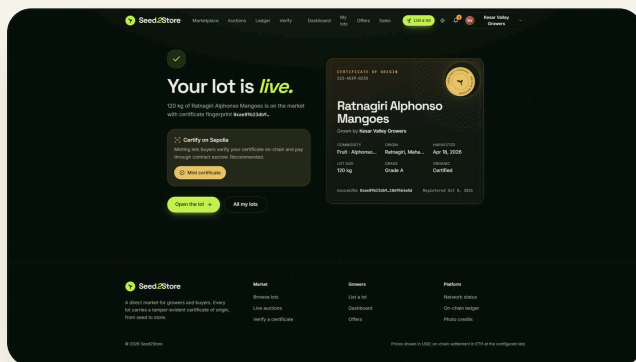
- 1 **Connect a wallet.** Any browser wallet is discovered via EIP-6963 (MetaMask, Rabby, Coinbase, Brave). Or pick **Burner wallet** to explore instantly with a throwaway key kept in your browser.
- 2 **Sign one message.** The server issues a nonce; you sign "Sign in to Seed2Store". It costs nothing and sends no transaction. The server verifies the signature and sets a signed session cookie for 7 days.
- 3 **Pick a role and a name.** Buyer or farmer, plus an optional display name and location under Profile. Counterparties trust a named grower over a bare address.

The grower's journey.

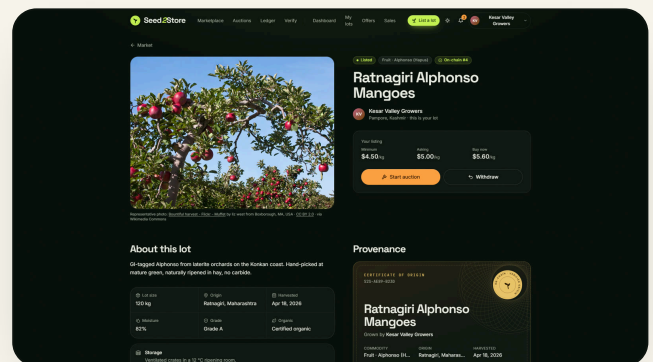
- 1 The crop.** Title, crop type, variety and a description: what a buyer would ask you on the phone.
- 2 Origin & quality.** Location, harvest date, grade, moisture, organic certification and storage.
- 3 Photos.** Drag and drop up to six. They're stored content-addressed by SHA-256 in MongoDB GridFS (or IPFS). Lots without photos get a generated field illustration.
- 4 Price & quantity.** A minimum (the floor for offers and auctions), an asking price, and an optional buy-now price per unit.



The certificate preview fills in live as you type.



Published. The provenance fields are sealed with a keccak-256 fingerprint and the lot gets its serial.

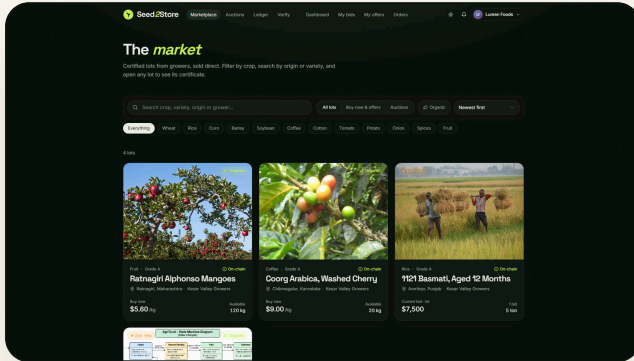


Minted. One click mints the ERC-721 certificate on Sepolia. The lot shows **ON-CHAIN #4**.

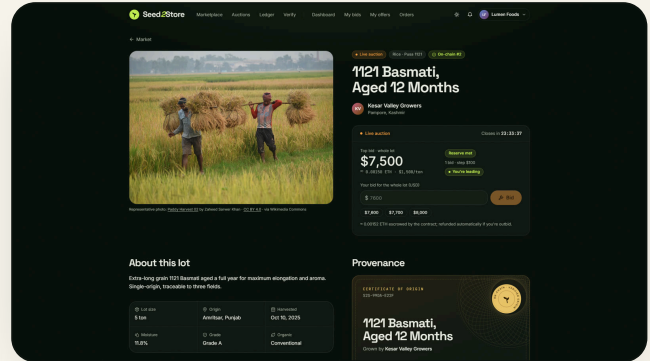
From the lot page the grower can **start an auction** (opening bid, optional reserve, 6 hours to 7 days), **withdraw** or **relist**, and answer offers. Certified lots trade whole and settle on-chain.

The buyer's journey.

Search across title, variety, origin and grower. Filter by crop and organic. Sort by price or by auctions ending soon.



MARKET Certified lots from real growers, with openly licensed photos.



A live on-chain auction (NFT #2). The bid is escrowed in the contract; the previous leader is refunded automatically.

Bid

Bids are for the whole lot. Quick-bid chips suggest the next valid amounts. On-chain auctions escrow your ETH and refund it automatically if you're outbid. A bid in the last 10 minutes adds 10 minutes.

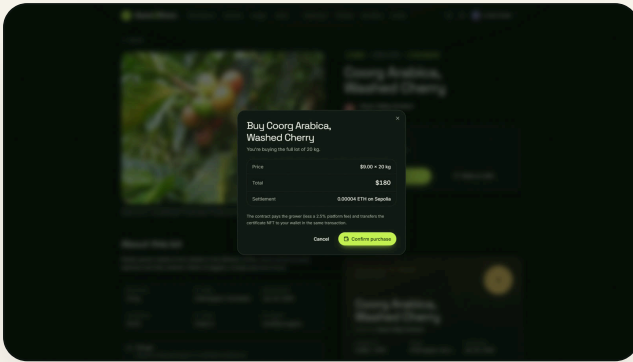
Offer

Propose a quantity and a price per unit, plus a note. The grower has 72 hours to accept, which opens an order and reserves the quantity, or to decline with a reply.

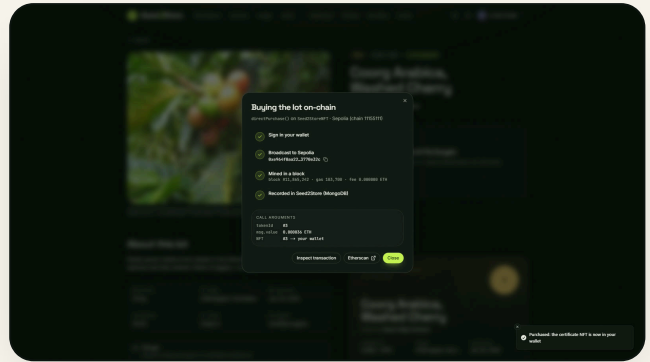
Buy now

A confirmation shows the total, the per-unit price and the settlement. For certified lots it's one `directPurchase` transaction.

The sale, and after.



Buying 12 kg of saffron: \$67,200, settled as 26.88 ETH on-chain.

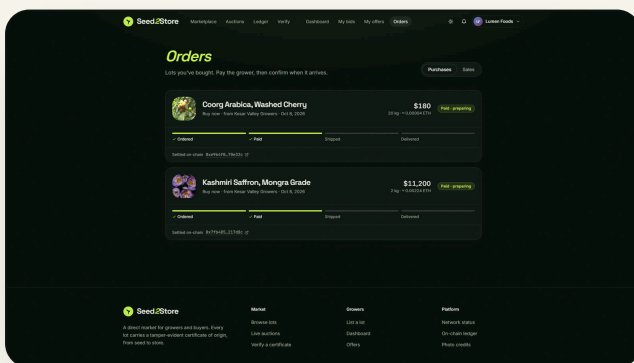


`directPurchase()` mined on Sepolia: block, gas and fee, then recorded in MongoDB. The NFT moves to the buyer.

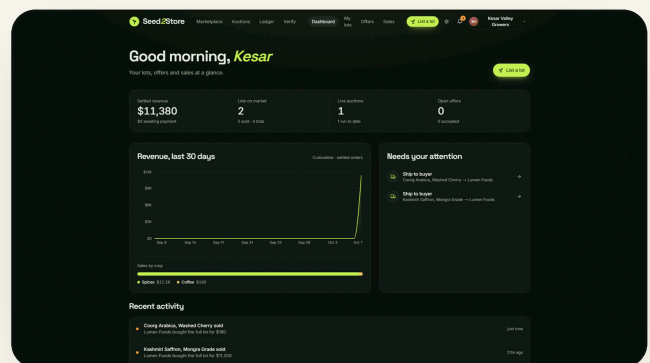
One transaction settles both sides

The contract pays the grower the buyout minus a 2.5% platform fee and transfers the certificate NFT to the buyer in the same transaction. The app records the transaction hash, marks the lot sold and opens a **paid** order.

ORDER STEP	WHO	WHAT HAPPENS
Ordered	automatic	Opened by buy-now, an accepted offer, or an auction closing above its reserve.
Paid	grower	Automatic for on-chain purchases; otherwise the grower confirms payment received.
Shipped	grower	Mark shipped. The buyer is notified.
Delivered	buyer	Confirm delivery closes the order. The grower is notified.
Cancelled	either	Only before payment. The goods return to the market.



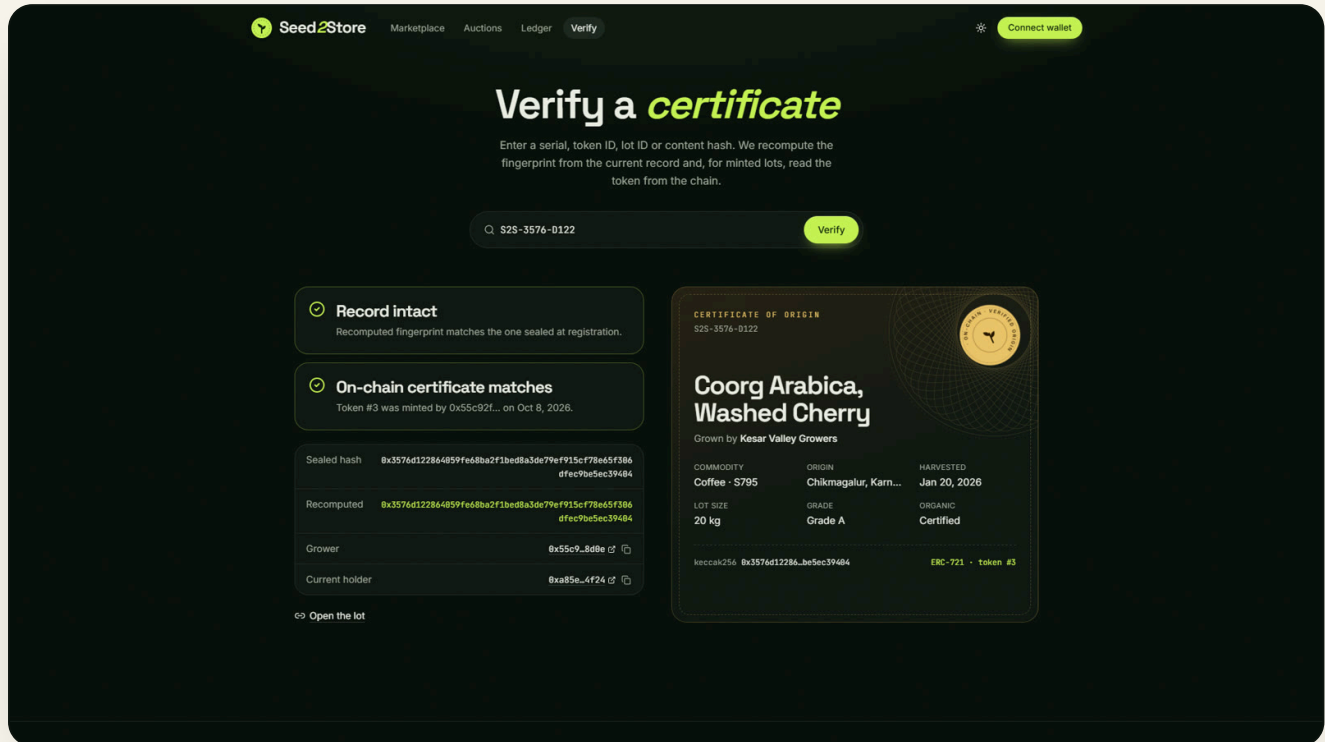
Buyer: **Paid** · **preparing**, with the on-chain transaction.



Grower dashboard: \$67,200 settled, 30-day revenue, "Ship to buyer".

Anyone can **check the papers.**

The certificate outlives the sale. Verification needs no wallet and no account.



CHECK 1

Record intact

The server recomputes

`keccak256(canonicalJSON(fields))` from the live record and compares it to the hash sealed at registration. Any edit to origin, harvest date, grade, photos, or the grower breaks the match.

CHECK 2

On-chain certificate matches

For minted lots it reads `getCropCertificate` and `ownerOf` from the chain: who issued the token, when, and who holds it now.

What the fingerprint covers

Title, description, crop type, variety, unit, harvest date, origin, organic flag, grade, moisture, storage, photo URLs, the grower's wallet and the registration time. **Quantity and prices are deliberately excluded:** they legitimately change as a lot trades.

Look-ups accept the serial (`s2s-xxxx-xxxx` , the first 8 hex digits of the hash), the token ID, the lot ID, the full hash, or the mint transaction hash.

How it's built.

Browser

Next.js 16 pages · React 19 ·
Tailwind v4 · SWR. Wallet layer:
EIP-6963 discovery + burner
wallet, ethers v6 contract hooks.

Route handlers

`app/api/*` : authenticate the
session, parse, call one service.
Thin by design.

services.ts

Every business rule, written once:
listing, auctions, offers, orders,
stats, verification, notifications.

Store

MongoDB (collections, indexes,
GridFS, meta) when
`MONGODB_URI` is set; otherwise
a local JSON file.

Seed2StoreNFT

ERC-721 certificates, escrowed
auctions and direct purchase on
any EVM chain.

Storage

Photos in GridFS (or IPFS via
Pinata). Token metadata from
`/api/metadata/:id` or IPFS.

Principles

- 1 **One rules engine.** Both storage backends run the identical service code, so there's no drift between a demo and production.
- 2 **Atomic transitions.** Buy-now, bids, offer acceptance, auction settlement and order steps use compare-and-set (`findOneAndUpdate` with the expected state in the filter). If two buyers click at once, exactly one wins; the other gets a 409.
- 3 **Chain first, then record.** For certified lots the wallet transaction happens first and the API records its hash. The API refuses off-chain shortcuts for those lots.
- 4 **Lazy housekeeping.** Ended auctions settle and stale offers expire on read, so there's no cron to run.
- 5 **Stateless sessions.** HMAC-SHA256 cookies work on serverless. The generated secret lives in MongoDB's `meta` collection unless `AUTH_SECRET` is set.

MongoDB & the contract.

COLLECTION	CONTENTS	INDEXES
users	wallet, role, display name, location, bio	wallet_address (unique), role
crops	lots: provenance, prices, status, content_hash, NFT linkage	status+created_at, farmer_id, content_hash, nft_token_id
auctions	ladder state, end time, reserve, chain ID, winner	status+end_time, crop_id
bids	every bid, optional tx hash	auction_id+amount, bidder_id
offers	quantity, price, message, reply, status	crop_id+status, buyer_id
orders	source, payment & delivery status, tx hash	buyer_id, farmer_id, created_at
notifications	per-user feed with deep links	user_id+created_at
uploads.*	GridFS photos, content-addressed by SHA-256	—
meta	generated session secret	—

Documents use the domain UUID as `_id`; timestamps are ISO-8601. Works with a local server or Atlas. A fresh database starts empty; `DEMO_SEED=true` loads a sample market for local demos.

Seed2StoreNFT · "Seed2Store Certificate" (S2SC)

FUNCTION	BEHAVIOUR
<code>createCropCertificate(...)</code>	Mints to the grower; stores key fields and the metadata URI.
<code>createAuction(id, start, reserve, step, duration)</code>	1 hour to 30 days; one live auction per token.
<code>placeBid(auctionId) payable</code>	Escrows the bid, refunds the previous leader; the last 10 minutes extend the end.
<code>finalizeAuction(auctionId)</code>	Pays the grower and transfers the NFT, or refunds and frees the lot for relisting.
<code>directPurchase(tokenId) payable</code>	Pays the grower minus 2.5%, transfers the NFT; blocked during live auctions.

OpenZeppelin 4.9 · Solidity 0.8.19 · `nonReentrant` on value transfers. Prices are entered in USD and converted at `NEXT_PUBLIC_ETH_USD`.

API reference.

All under `/api`. Writes need a session; market reads are public. Errors return `{ "error": "..." }` with 400 / 401 / 403 / 404 / 409.

METHOD	ROUTE	PURPOSE
GET	<code>/auth/nonce?address=</code>	Sign-in message to sign
POST	<code>/auth/verify</code>	{ address, signature } → session cookie
GET · POST	<code>/auth/me</code> · <code>/auth/logout</code>	Current user · end session
PATCH	<code>/me</code>	Role, display name, location, bio
GET · POST	<code>/crops</code>	Market search (q, crop_type, organic, price, status, sort, page) · register a lot
GET	<code>/crops/mine</code>	Your lots
GET · PATCH	<code>/crops/:id</code>	Lot detail · delist / relist
POST	<code>/crops/:id/mint</code> · <code>/crops/:id/buy</code>	Record a mint · buy now
GET	<code>/crops/:id/offers</code>	Offers on a lot (role-scoped)
GET · POST	<code>/auctions</code>	List (active / ended) · create
GET · DELETE	<code>/auctions/:id</code>	Auction + bids · cancel (no bids, off-chain)
POST	<code>/auctions/:id/bids</code> · <code>/finalize</code>	Place a bid · record on-chain settlement
GET	<code>/bids</code>	Your bids with standing
GET · POST	<code>/offers</code>	Received / sent · make an offer
PATCH	<code>/offers/:id</code>	accept · reject · withdraw (+ message)
GET · PATCH	<code>/orders</code> · <code>/orders/:id</code>	Purchases / sales · confirm_payment, ship, deliver, cancel
GET · PATCH	<code>/notifications</code>	Feed · mark read
GET	<code>/stats</code> · <code>/stats/market</code>	Dashboard stats · market totals
POST · GET	<code>/uploads</code> · <code>/uploads/:file</code>	Upload a photo · serve it
GET	<code>/metadata/:id</code> · <code>/verify?q=</code> · <code>/status</code>	Token metadata · verification · system status

Setup, config & deploy.

Zero config

```
npm install
npm run dev # → localhost:3000
```

MongoDB

```
# .env.local
MONGODB_URI=mongodb://127.0.0.1:27017
MONGODB_DB=seed2store
```

Blockchain

```
npm run chain # Hardhat :8545
npm run deploy:local # writes
.env.local
npm run dev
```

Tests

```
npm test # suite runs on JSON + real
MongoDB
npm run typecheck && npm run build
```

Environment

VARIABLE	PURPOSE
MONGODB_URI / _DB	Use MongoDB
MONGODB_SEED	Seed an empty DB (default true)
NEXT_PUBLIC_NFT_CONTRACT	Enable on-chain flows
NEXT_PUBLIC_CHAIN_ID	31337 · 1337 · 11155111
NEXT_PUBLIC_RPC_URL	RPC for burner + reads
NEXT_PUBLIC_EXPLORER_URL	Tx and address links
NEXT_PUBLIC_ETH_USD	USD per ETH (2500)
PINATA_JWT	IPFS for photos + metadata
AUTH_SECRET	Session key (else stored in DB)
PRIVATE_KEY	Contract deployment

Deploy: Vercel + Atlas

- 1 Create an Atlas cluster; set `MONGODB_URI`.
- 2 Set `AUTH_SECRET`.
- 3 `npm run deploy:sepolia`; set the chain variables.
- 4 Optionally set `PINATA_JWT`.

Security & troubleshooting.

Authentication

A signed message with a nonce held in a short-lived signed cookie. The signature is recovered server-side. Sessions are HMAC cookies: httpOnly, sameSite=lax, secure in production.

Authorization

Every write checks the session user: owners for lots, auctions and offers; parties for orders. Client-sent wallet addresses are never trusted.

Integrity

Compare-and-set on every racy transition, a unique wallet index, and certified lots that can't be split or settled off-chain.

Uploads & contract

Images only, 8 MB cap, sha256-named files (no path traversal). The contract uses reentrancy guards, refunds on outbid, and no direct purchase during auctions.

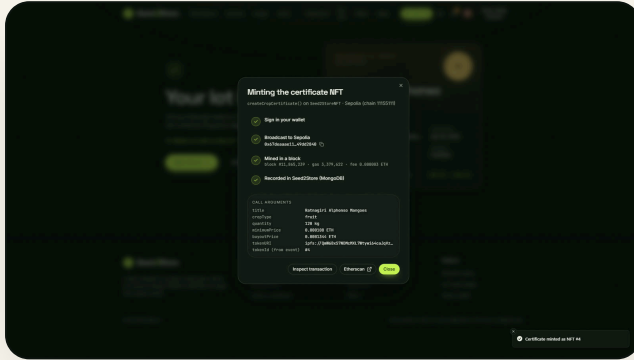
Troubleshooting

SYMPTOM	FIX
Peer-dependency errors on install	Keep the repo's <code>.npmrc</code> (legacy-peer-deps).
<code>/status</code> : MongoDB unreachable	Check <code>MONGODB_URI</code> , that <code>mongod</code> is running, or the Atlas IP allow-list.
"No contract code at ..."	The Hardhat node restarted. Run <code>npm run deploy:local</code> again and restart dev.
MetaMask "nonce too high"	Settings → Advanced → Clear activity tab data.
Signed out after changing <code>AUTH_SECRET</code>	Expected. Connect again.
Demo auctions all ended	<code>npm run db:reset</code> , then restart. Seed times are relative to now.

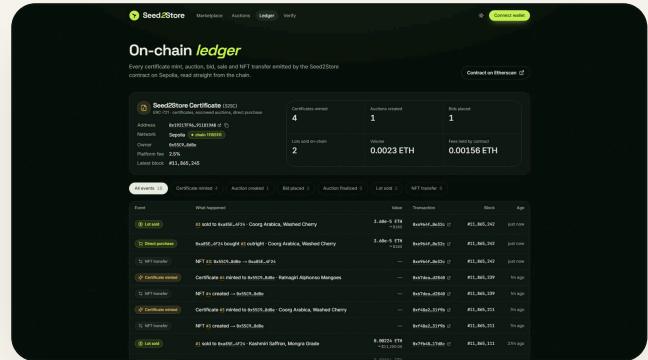
Every transaction, on view.

Seed2StoreNFT (S2SC) is live on Sepolia at

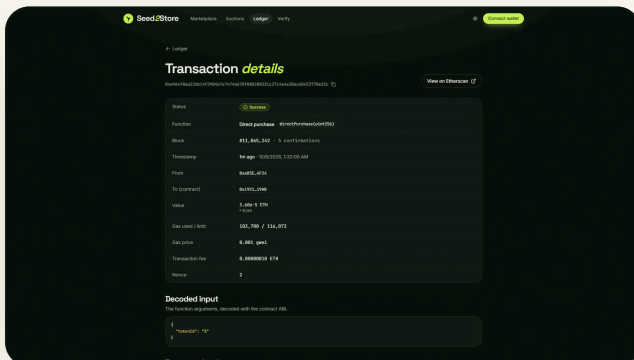
0x19217F965bB80B88DBb8e66dc424414A911D19AB, verified on Sourcify. Every write goes through a tracker (sign, broadcast, mined, recorded), and the app ships its own explorer.



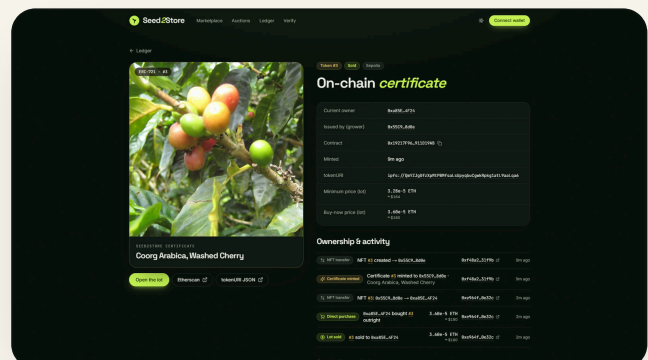
Mint. `createCropCertificate()` with its call arguments, block, gas and fee.



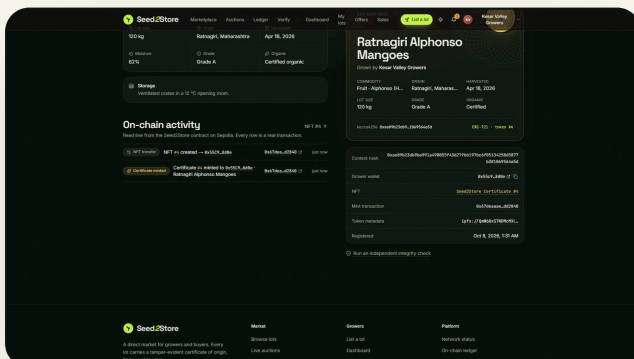
Ledger. Every mint, auction, bid, sale and transfer, decoded from the contract's events.



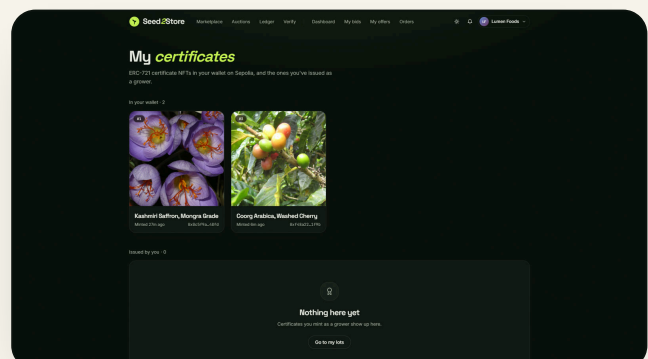
Transaction. Decoded input, value, gas, fee and emitted events, with an Etherscan link.



NFT #3. Owner, issuer, IPFS tokenURI (Pinata) and full ownership history.



On-chain activity on every lot page.



My certificates. NFTs held in the connected MetaMask or burner wallet.

Seed2Store

Every harvest, **on the record.**

```
$ npm install && npm run dev
```

SEPOLIA · MONGODB ATLAS · PINATA IPFS · ERC-721 CERTIFICATES